

Development Environment Setup

Yunmin Go

School of AICEE

Handong Global University

Procedure

- Step 1. Install Docker Desktop
- Step 2. Create Docker Image
- Step 3. Start Docker Containers
- Step 4. Install VSCode
- Step 5. Install Extensions in VSCode
- Step 6. Connect to Containers
- Step 7. Compile and Test

Read the instructions carefully and follow the installation steps. You can do it!
There are trouble shooting slides after Step 7, so please refer to them if needed.

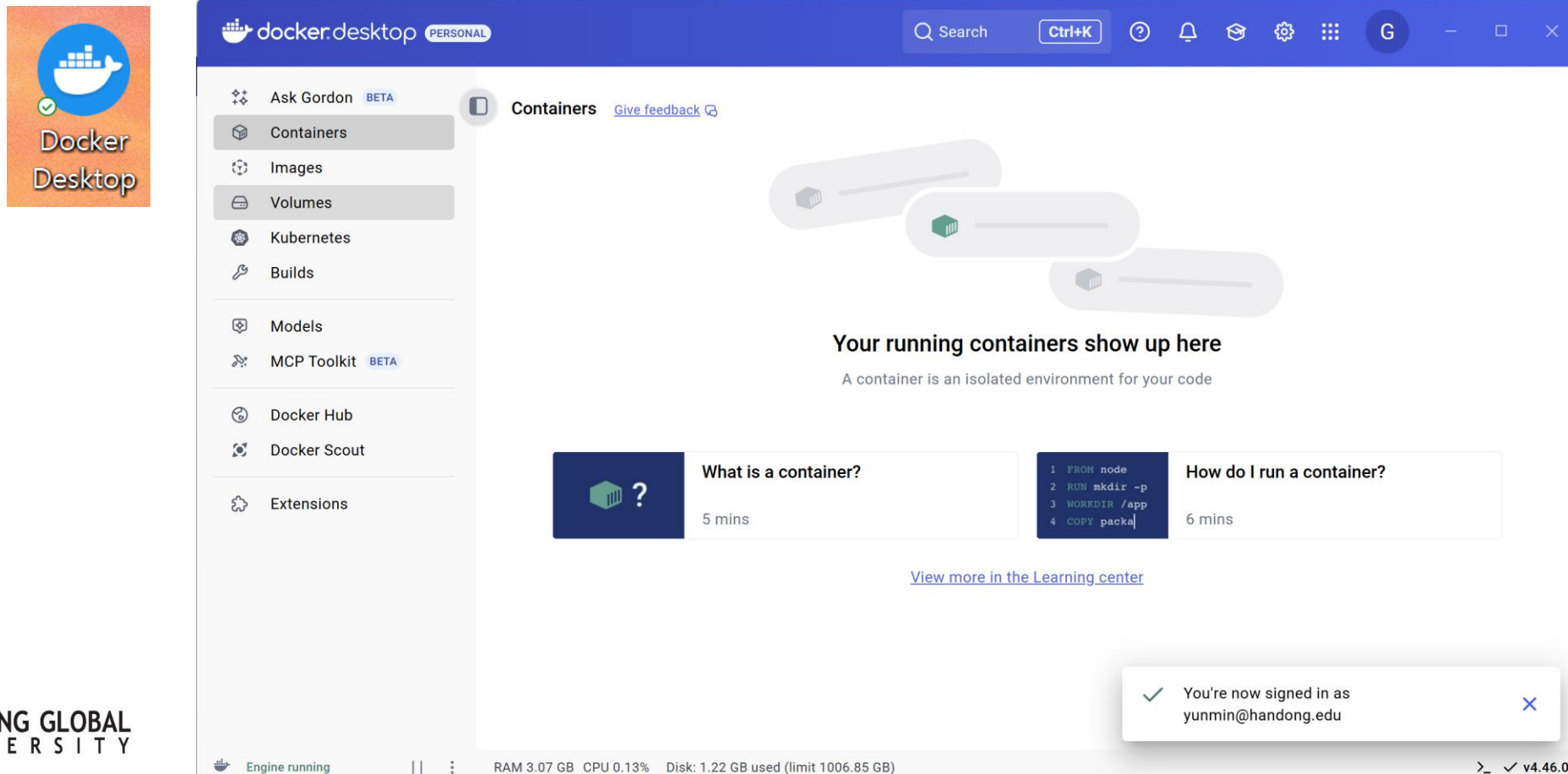
Step 1. Install Docker Desktop

- Please refer to the website below to install Docker Desktop
 - For Windows user: <https://docs.docker.com/desktop/install/windows-install/>
 - Please carefully check the 'System requirements' on the web site. You might need to install WSL 2.
 - For Mac user: <https://docs.docker.com/desktop/setup/install/mac-install/>

The screenshot shows the Docker Docs website. The top navigation bar includes 'dockerdocs', 'Get started', 'Guides', 'Manuals', and 'Reference'. The left sidebar lists various topics under 'PRODUCTS', including 'Docker Desktop', 'Setup', 'Install', 'Mac', 'Windows', 'Linux', and 'VM or VDI environments'. The main content area is titled 'Install Docker Desktop on Windows' and includes a 'Docker Desktop terms' section. Below this, there are download links for 'Docker Desktop for Windows - x86_64', 'Docker Desktop for Windows - x86_64 on the Microsoft Store', and 'Docker Desktop for Windows - Arm (Early Access)'. A 'System requirements' section is highlighted with a red box. The right sidebar contains a 'Table of contents' with links to 'System requirements', 'Administrator privileges and installation requirements', 'WSL: Verification and setup', 'Option 1: Install or update WSL via the terminal', 'Option 2: Install WSL via the MSI package', 'Install Docker Desktop on Windows', 'Install interactively', 'Install from the command line', 'Start Docker Desktop', and 'Where to go next'.

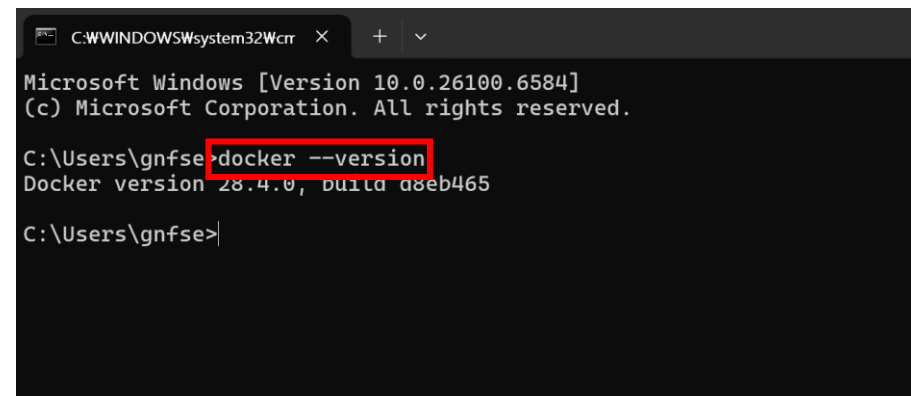
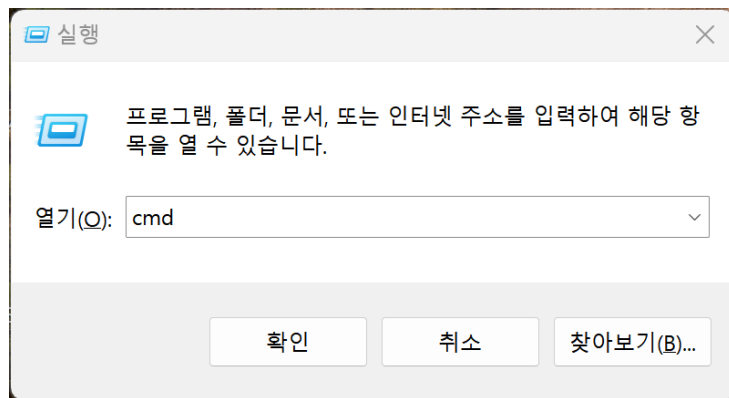
Step 1. Install Docker Desktop

- If you succeed, you can see the windows as shown below after sign in. You might create an account if you don't have.



Step 1. Install Docker Desktop

- For Windows users, press the 'Windows key + r' to open the Run dialog, then type 'cmd' to launch the 'Command Prompt'. If the Docker Desktop installation was successful, you can check the Docker version by running 'docker --version', and the version will appear as shown in the image below.
- For Mac users, open 'Terminal'. The rest is the same as for Windows users.



Step 2. Create Docker Image

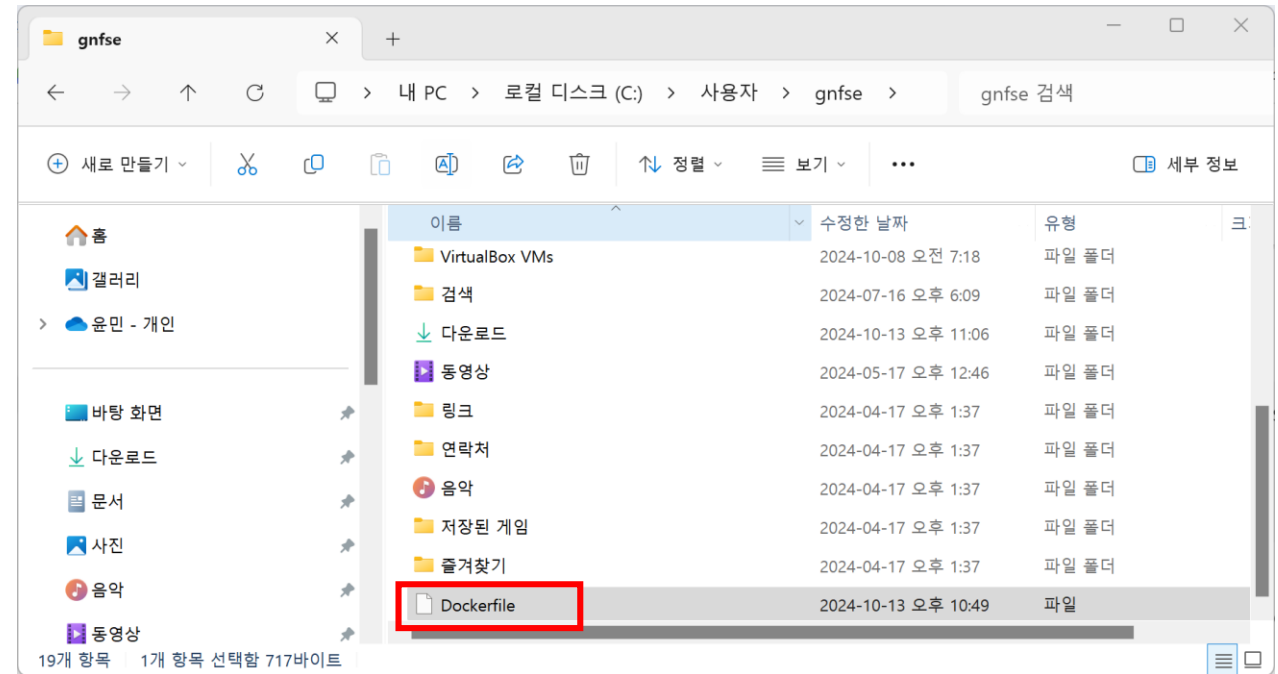
- Copy the 'Dockerfile' downloaded from LMS to the home directory where the Command Prompt is launched.

```
C:\WINDOWS\system32\cmd.exe
Microsoft Windows [Version 10.0.26100.6584]
(c) Microsoft Corporation. All rights reserved.

C:\Users\gnfse>docker --version
Docker version 28.4.0, build d8eb465

C:\Users\gnfse>
```

In this example, the Dockerfile is copied into 'C:\Users\gnfse'.



Step 2. Create Docker Image

- Run the command below to build the Docker image.
 - > docker build -t hgu-comnet . ← We need a dot here!

```
C:\WINDOWS\system32\cmd.exe
Microsoft Windows [Version 10.0.26100.6584]
(c) Microsoft Corporation. All rights reserved.

C:\Users\gnfse>docker --version
Docker version 28.4.0, build d8eb465

C:\Users\gnfse>
C:\Users\gnfse>docker build -t hgu-comnet .
[+] Building 6.1s (5/7)
=> [internal] load build definition from Dockerfile
=> => transferring dockerfile: 632B
=> [internal] load metadata for docker.io/library/ubuntu:latest
=> [auth] library/ubuntu:pull token for registry-1.docker.io
=> [internal] load .dockerignore
=> => transferring context: 2B
=> [1/3] FROM docker.io/library/ubuntu:latest@sha256:9cbcd754112939e914291337b5e554b07ad7c392491dba6daf25eef1332 1.9s
=> => resolve docker.io/library/ubuntu:latest@sha256:9cbcd754112939e914291337b5e554b07ad7c392491dba6daf25eef1332 0.0s
=> => sha256:76249c7cd50397d2e8c06a75106723d057deaba0ffbc7f4af1bb02bcf71d81cf 29.72MB / 29.72MB 1.3s
=> => extracting sha256:76249c7cd50397d2e8c06a75106723d057deaba0ffbc7f4af1bb02bcf71d81cf 0.5s
=> [2/3] RUN apt-get update && apt-get install -y gcc make vim net-tools iputils-ping ip 0.6s
docker:desktop-linux
```

Step 2. Create Docker Image

- After building the Docker image, you can check the created Docker image by running the following command
 - > docker images

```
C:\Users\gnfse>docker build -t hgu-comnet .
[+] Building 51.9s (8/8) FINISHED
=> [internal] load build definition from Dockerfile
=> => transferring dockerfile: 632B
=> [internal] load metadata for docker.io/library/ubuntu:latest
=> [auth] library/ubuntu:pull token for registry-1.docker.io
=> [internal] load .dockerignore
=> => transferring context: 2B
=> [1/3] FROM docker.io/library/ubuntu:latest@sha256:9cbcd754112939e914291337b5e554b07ad7c392491dba6daf25eef1332
=> => resolve docker.io/library/ubuntu:latest@sha256:9cbcd754112939e914291337b5e554b07ad7c392491dba6daf25eef1332
=> => sha256:76249c7cd50397d2e8c06a75106723d057deaba0ffbc7f4af1bb02bcf71d81cf 29.72MB / 29.72MB
=> => extracting sha256:76249c7cd50397d2e8c06a75106723d057deaba0ffbc7f4af1bb02bcf71d81cf
=> [2/3] RUN apt-get update && apt-get install -y gcc make vim net-tools iputils-ping
=> [3/3] WORKDIR /root
=> exporting to image
=> => exporting layers
=> => exporting manifest sha256:cf8ddd004aaf556733e333db7a961b441d13724dad656fe41c74e1e3c7de3046
=> => exporting config sha256:efa4d7b126ebbe24fc005a19069accacc6e7acbeda81b0f2e1ce857291cc62
=> => exporting attestation manifest sha256:fd225f6883d018c0e7a62b7f1df3e86f60c43bdd87fb18d85ebf5870457801d2
=> => exporting manifest list sha256:58bce93ea901b2a6a6b046bbaade9c9b660605013dc758650d48e0321cc442b1
=> => naming to docker.io/library/hgu-comnet:latest
=> => unpacking to docker.io/library/hgu-comnet:latest

View build details: docker-desktop://dashboard/build/desktop-linux/desktop-linux/cif8h2ckzsi983pr62uq4u76g

C:\Users\gnfse>docker images
```

REPOSITORY	TAG	IMAGE ID	CREATED	SIZE
hgu-comnet	latest	58bce93ea901	22 seconds ago	742MB

```
C:\Users\gnfse>
```

Step 3. Start Docker Containers

- Run the two commands below to start two Docker containers.

- > docker run -d -it --name container1 -p 5000:9000 hgu-comnet

- > docker run -d -it --name container2 -p 6000:9000 hgu-comnet

Container's name

Host computers' port

You can change the container's name and the host computer's port as you like.

- You can check the currently running containers by running

- > docker ps ('docker ps -a' shows all containers, including exited ones)

```
C:\WINDOWS\system32\cmd.exe
View build details: docker-desktop://dashboard/build/desktop-linux/desktop-linux/cif8h2ckzsi983pr62uq4u76g

C:\Users\gnfse>docker images
REPOSITORY    TAG       IMAGE ID       CREATED        SIZE
hgu-comnet    latest    58bce93ea901   22 seconds ago 742MB

C:\Users\gnfse>docker run -d -it --name container1 -p 5000:9000 hgu-comnet
3ee2fc834ddd908c7c65ff392cc99f3f4289d52a17963a4528016f32720198f6

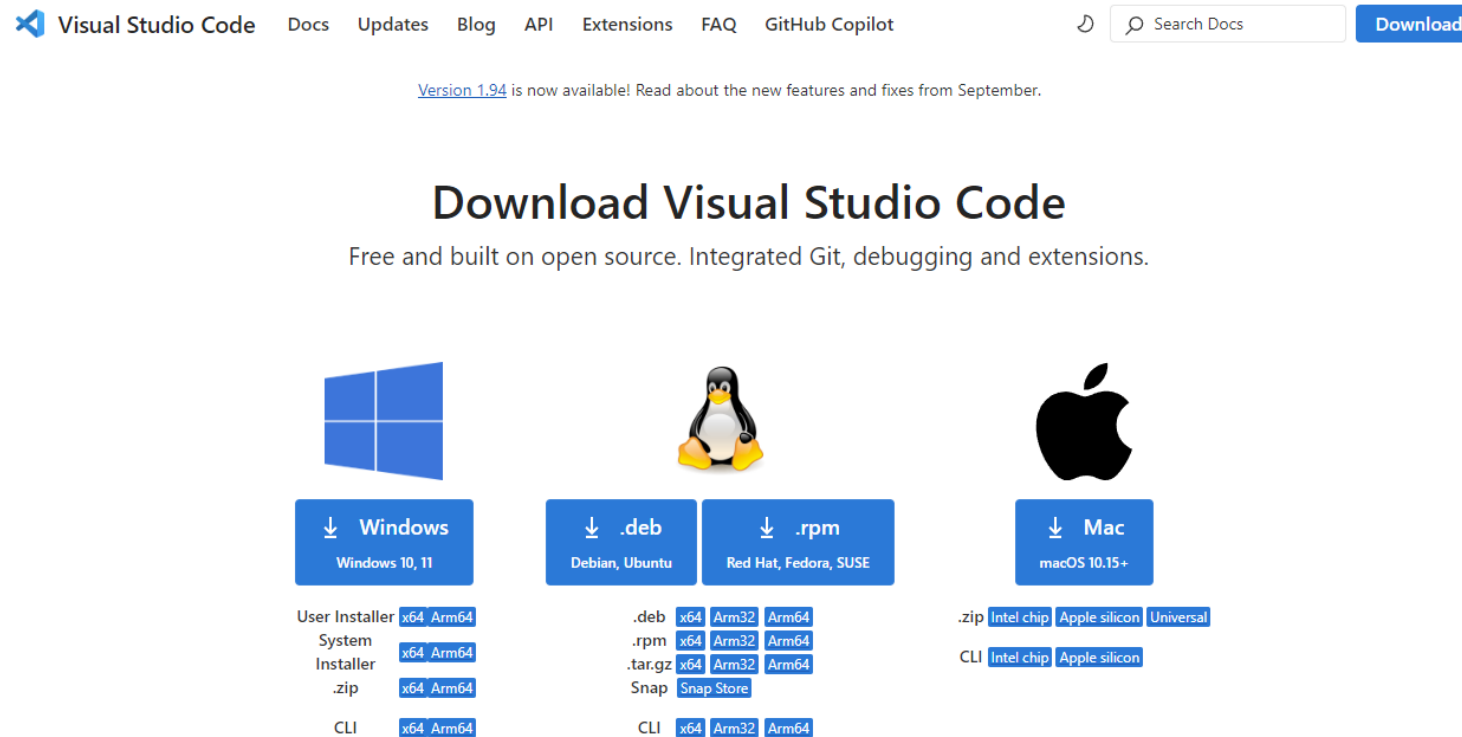
C:\Users\gnfse>docker run -d -it --name container2 -p 6000:9000 hgu-comnet
104f345eaf61c591b76a4ae3bdd3d709b3e0b22b57e57edb5e51aa535837e4ab

C:\Users\gnfse>docker ps
CONTAINER ID   IMAGE          COMMAND                  CREATED        STATUS        PORTS                                                                 NAMES
104f345eaf61   hgu-comnet    "/bin/bash"             3 seconds ago Up 2 seconds   0.0.0.0:6000->9000/tcp, [::]:6000->9000/tcp   container2
3ee2fc834ddd   hgu-comnet    "/bin/bash"             17 seconds ago Up 16 seconds  0.0.0.0:5000->9000/tcp, [::]:5000->9000/tcp   container1

C:\Users\gnfse>
C:\Users\gnfse>
```

Step 4. Install VSCode

- Download Visual Studio Code (VSCode) and install it.
 - <https://code.visualstudio.com/Download>



The screenshot shows the Visual Studio Code download page. At the top, there's a navigation bar with links for Visual Studio Code, Docs, Updates, Blog, API, Extensions, FAQ, and GitHub Copilot. A search bar and a 'Download' button are also present. Below the navigation bar, a message states 'Version 1.94 is now available! Read about the new features and fixes from September.' The main heading is 'Download Visual Studio Code', followed by the tagline 'Free and built on open source. Integrated Git, debugging and extensions.' The page is divided into three main sections for Windows, Linux, and Mac. Each section has a download button and a list of available installers and their architectures.

Visual Studio Code Docs Updates Blog API Extensions FAQ GitHub Copilot Search Docs Download

Version 1.94 is now available! Read about the new features and fixes from September.

Download Visual Studio Code

Free and built on open source. Integrated Git, debugging and extensions.



↓ Windows
Windows 10, 11

User Installer	x64	Arm64
System Installer	x64	Arm64
.zip	x64	Arm64
CLI	x64	Arm64



↓ .deb
Debian, Ubuntu

↓ .rpm
Red Hat, Fedora, SUSE

.deb	x64	Arm32	Arm64
.rpm	x64	Arm32	Arm64
.tar.gz	x64	Arm32	Arm64
Snap	Snap Store		
CLI	x64	Arm32	Arm64

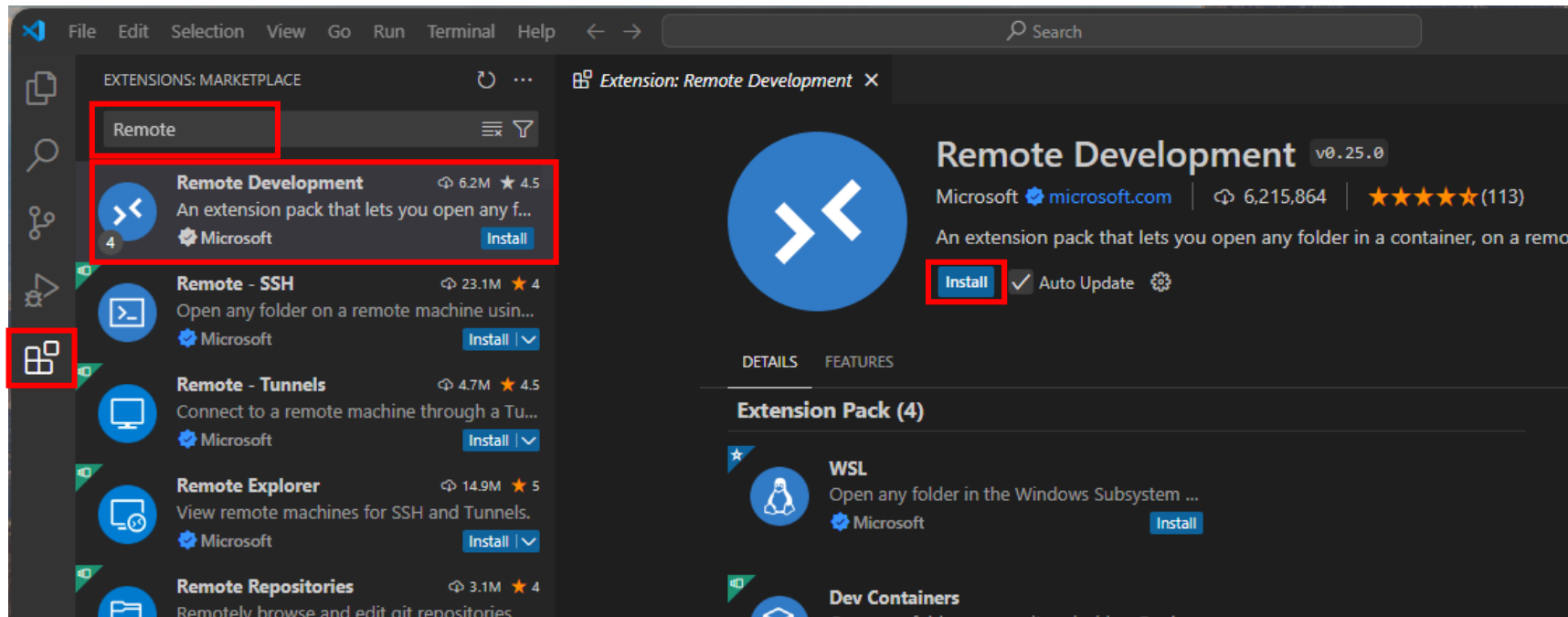


↓ Mac
macOS 10.15+

.zip	Intel chip	Apple silicon	Universal
CLI	Intel chip	Apple silicon	

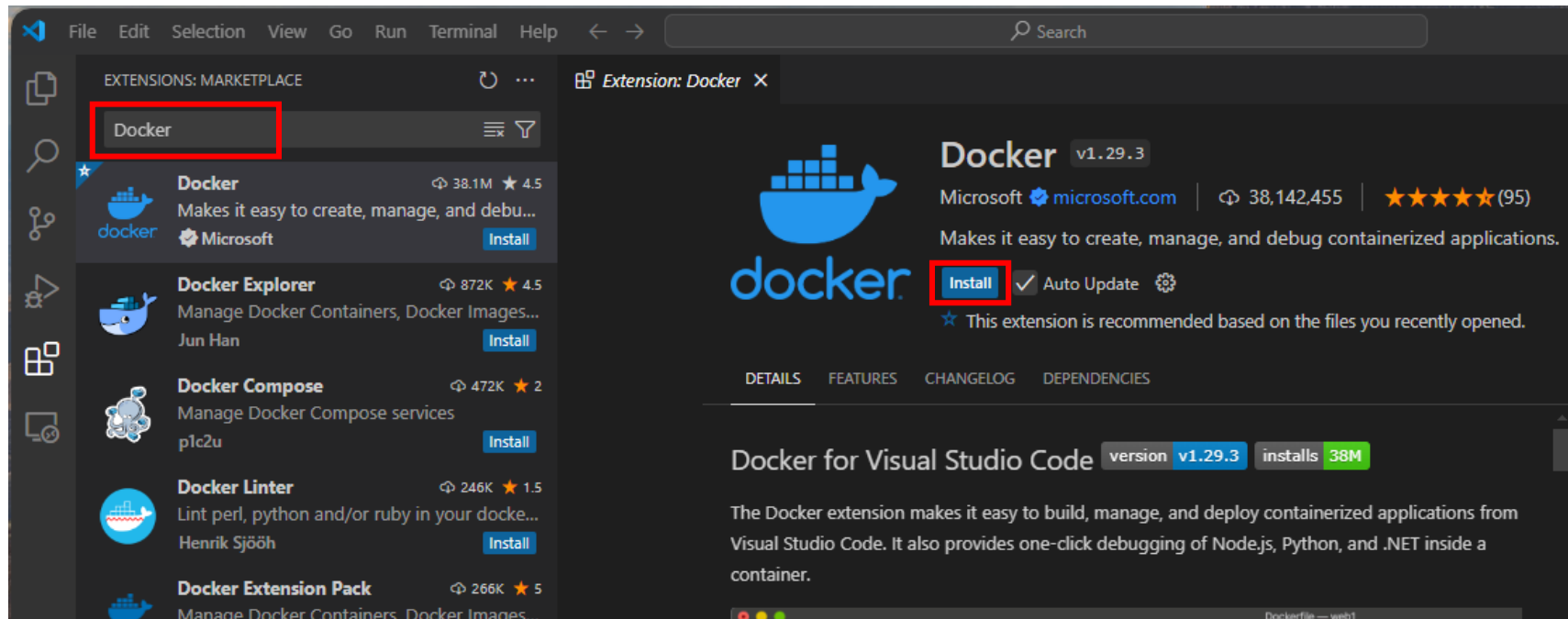
Step 5. Install Extensions in VSCode

- After launching VSCode, search for 'Remote Development' in the Extensions section, and install it.



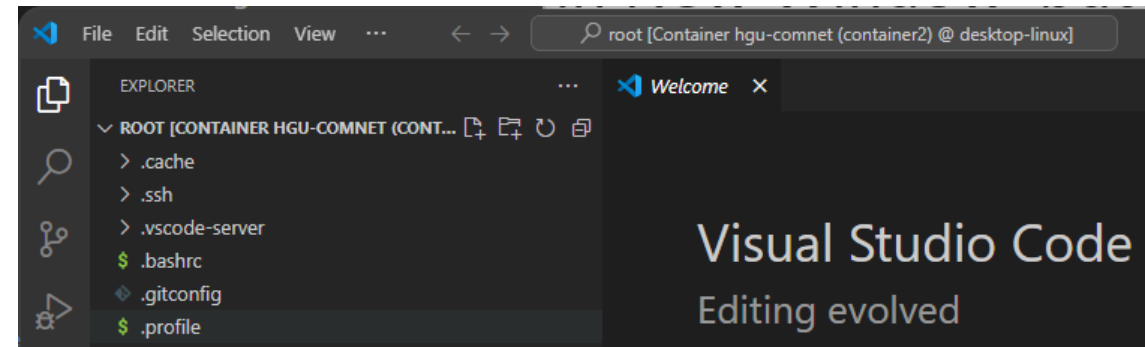
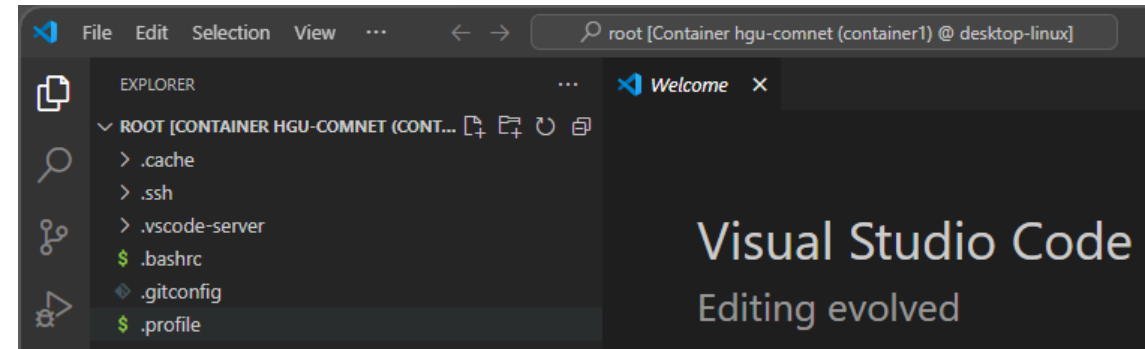
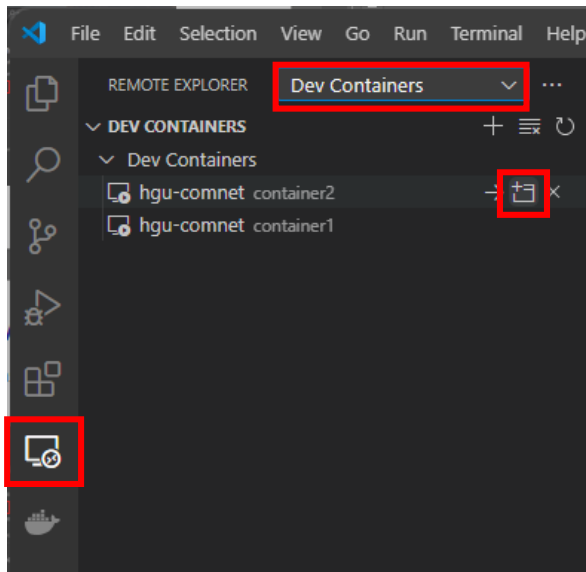
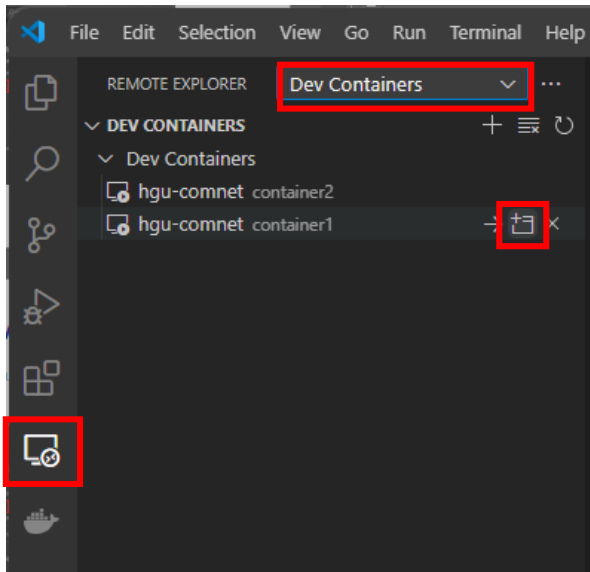
Step 5. Install Extensions in VSCode

- Now, search for 'Docker' in the Extensions section, and install it.



Step 6. Connect to Containers

- In the 'Remote Explorer', select the 'Dev Containers'. Then click the 'Attach in New Window' button for both container1 and container2 to open them in a new window.



Step 7. Compile and Test

- In VSCode for both container1 and container2, open the terminal and use the ifconfig command to check the IP address.

```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS
● root@3ee2fc834ddd:~# ifconfig
eth0: flags=4163<UP,BROADCAST,RUNNING,MULTICAST> mtu 1500
    inet 172.17.0.2 netmask 255.255.0.0 broadcast 172.17.255.255
    ether 5e:56:f4:21:50:65 txqueuelen 0 (Ethernet)
    RX packets 13877 bytes 20108345 (20.1 MB)
    RX errors 0 dropped 0 overruns 0 frame 0
    TX packets 2676 bytes 310501 (310.5 KB)
    TX errors 0 dropped 0 overruns 0 carrier 0 collisions 0

lo: flags=73<UP,LOOPBACK,RUNNING> mtu 65536
    inet 127.0.0.1 netmask 255.0.0.0
    inet6 ::1 prefixlen 128 scopeid 0x10<host>
    loop txqueuelen 1000 (Local Loopback)
    RX packets 3267 bytes 38678335 (38.6 MB)
    RX errors 0 dropped 0 overruns 0 frame 0
    TX packets 3267 bytes 38678335 (38.6 MB)
    TX errors 0 dropped 0 overruns 0 carrier 0 collisions 0
```

```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS
● root@104f345eaf61:~# ifconfig
eth0: flags=4163<UP,BROADCAST,RUNNING,MULTICAST> mtu 1500
    inet 172.17.0.3 netmask 255.255.0.0 broadcast 172.17.255.255
    ether 1e:ae:68:c3:a9:d0 txqueuelen 0 (Ethernet)
    RX packets 13862 bytes 20102576 (20.1 MB)
    RX errors 0 dropped 0 overruns 0 frame 0
    TX packets 2836 bytes 320385 (320.3 KB)
    TX errors 0 dropped 0 overruns 0 carrier 0 collisions 0

lo: flags=73<UP,LOOPBACK,RUNNING> mtu 65536
    inet 127.0.0.1 netmask 255.0.0.0
    inet6 ::1 prefixlen 128 scopeid 0x10<host>
    loop txqueuelen 1000 (Local Loopback)
    RX packets 3206 bytes 38637852 (38.6 MB)
    RX errors 0 dropped 0 overruns 0 frame 0
    TX packets 3206 bytes 38637852 (38.6 MB)
    TX errors 0 dropped 0 overruns 0 carrier 0 collisions 0
```

Step 7. Compile and Test

- In container1, write the tcp_server.c file, and in container2, write the tcp_client.c file.
- After the compilation using the commands below, run the server first, followed by the client, to check the result.

```
● root@9ed57e942644:~# gcc tcp_server.c -o tcp_server
● root@9ed57e942644:~# ./tcp_server 9190
○ root@9ed57e942644:~#
```

```
● root@d997512e884e:~# gcc tcp_client.c -o tcp_client
● root@d997512e884e:~# ./tcp_client 172.17.0.2 9190
Message from server: Hello World!
Function read call count: 13
○ root@d997512e884e:~#
```

Container1's IP Address
and tcp_server's port

If you can see this message, you are succeed.

Useful Docker Command

- Check the existing Docker images
 - > docker images
- Remove Docker images
 - > docker rmi <Image name or Image ID>
- Check the existing Docker containers
 - > docker ps ← Show only the running containers
 - > docker ps -a ← Show all containers including both running and stopped containers
- Start Docker container
 - > docker start <Container's name or ID>
- Stop Docker container
 - > docker stop <Container's name or ID>
- Remove Docker container
 - > docker rm <Container's name or ID>

Additional Slides for Trouble Shooting

Trouble Shooting

Issue: “zsh: command not found: docker” error when you check docker version in Step 1.

- In the Terminal,
 1. % vi ~/.zshrc
 2. Add docker path and exit (please check your docker directory before add)

```
export PATH="/Applications/Docker.app/Contents/Resources/bin:$PATH"
```
 3. % source ~/.zshrc
 4. % docker --version

Trouble Shooting

Issue: “permission denied” error when you build docker image in Step 2

```
ERROR: failed to solve: failed to read dockerfile: error from sender: failed to xattr .mysql_history  
: permission denied
```

Check file name

- In the Terminal,
 1. % ls -la ~/.mysql_history
 2. % sudo chmod 644 ~/.mysql_history
 3. % docker build -t hgu-comnet .

Trouble Shooting

Issue: “Ports are not available: ~~ bind: address already in use” error when you start docker container in Step 3.

```
docker: Error response from daemon: Ports are not available: exposing port TCP 0.0.0.0:5000 -> 0.0.0.0:0: listen tcp 0.0.0.0:5000: bind: address already in use.
```

- Change the host computer's port number as shown below

```
> docker run -d -it --name container1 -p 5000:9000 hgu-comnet
```

```
> docker run -d -it --name container2 -p 6000:9000 hgu-comnet
```



```
> docker run -d -it --name container1 -p 8088:9000 hgu-comnet
```

```
> docker run -d -it --name container2 -p 8089:9000 hgu-comnet
```

- You can also change the container's name e.g., hgnet1, hgnet2...

Trouble Shooting

Issue: “The container name “/container2” is already in user by ...”
error when you start docker container in Step 3.

1. Remove the existing container, name “container2”.
(assume that the existing container’s name is ‘container2’)

```
> docker rm container2
```

2. Start the docker

```
> docker run -d -it --name container2 -p 6000:9000 hgu-comnet
```

Please check the slide of “Useful Command for Docker”